

Practical Step-by-Step Example: MLP Forward, Backward & Adam Update

Reference Material for Defense

Multilayer Perceptron Project

September 11, 2026

Contents

1	Architecture & Initial Setup	2
2	Step 1: Forward Pass	3
2.1	Layer 1: Pre-activations Z^1 and Activations A^1	3
2.2	Layer 2: Pre-activations Z^2 and Output A^2	4
2.3	Loss Computation (Categorical Cross-Entropy)	5
3	Step 2: Backward Pass & Chain Rule	7
3.1	Proof: Simplification of Softmax with Categorical Cross-Entropy	7
3.2	Proof: Derivation of Error Propagation (δ^l) and Bias Gradient ($\frac{\partial L}{\partial b^l}$)	8
3.3	Output Layer Gradient (δ^2)	9
3.4	Weight and Bias Gradients at Layer 2 (dW^2, db^2)	10
3.5	Hidden Layer Error Projection (δ^1)	11
3.6	Weight and Bias Gradients at Layer 1 (dW^1, db^1)	11
4	Step 3: Adam Optimizer Step	13
4.1	First and Second Moment Accumulation (m_t, v_t)	13
4.2	Bias Correction ($\hat{m}_t, \hat{v}_t$)	13
4.3	Final Parameter Update	14

1 Architecture & Initial Setup

Network Topology

- **Input Layer** ($l = 0$): $n_0 = 4$ features.
- **Hidden Layer** ($l = 1$): $n_1 = 3$ neurons (Activation: ReLU).
- **Output Layer** ($l = 2$): $n_2 = 2$ neurons (Activation: Softmax).
- **Batch Size** (m): 2 normalized samples.

Initial Data Matrices

Input Matrix X (4×2):

$$X = A^0 = \begin{bmatrix} 0.5 & -0.2 \\ 0.8 & 0.4 \\ -0.1 & 0.9 \\ 0.3 & -0.6 \end{bmatrix}$$

We consider two samples, each with four features, in order to illustrate the complete process using a small mini-batch. The process for a single sample would be analogous, starting with A^0 represented as a 4×1 column vector.

Layer 1 Parameters ($W^1 \in \mathbb{R}^{3 \times 4}$, $b^1 \in \mathbb{R}^{3 \times 1}$):

$$W^1 = \begin{bmatrix} 0.35 & -0.50 & 0.20 & -0.10 \\ -0.25 & 0.40 & -0.30 & 0.15 \\ 0.10 & 0.05 & 0.60 & -0.45 \end{bmatrix}, \quad b^1 = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

Layer 2 Parameters ($W^2 \in \mathbb{R}^{2 \times 3}$, $b^2 \in \mathbb{R}^{2 \times 1}$):

$$W^2 = \begin{bmatrix} 0.40 & -0.30 & 0.50 \\ -0.10 & 0.60 & -0.20 \end{bmatrix}, \quad b^2 = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Target Labels Y (One-Hot, 2×2):

$$Y = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

2 Step 1: Forward Pass

2.1 Layer 1: Pre-activations Z^1 and Activations A^1

To process the mini-batch of size $m = 2$, we compute the matrix multiplication $W^1 A^0$ and add the bias vector b^1 via broadcasting:

$$Z^1 = W^1 A^0 + b^1$$

Substituting the numerical matrices:

$$Z^1 = \begin{bmatrix} 0.35 & -0.50 & 0.20 & -0.10 \\ -0.25 & 0.40 & -0.30 & 0.15 \\ 0.10 & 0.05 & 0.60 & -0.45 \end{bmatrix} \begin{bmatrix} 0.5 & -0.2 \\ 0.8 & 0.4 \\ -0.1 & 0.9 \\ 0.3 & -0.6 \end{bmatrix} + \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

Evaluating each element $(Z^1)_{ij}$:

- **Sample 1 (Column 1):**

- $Z_{11}^1 = (0.35)(0.5) + (-0.50)(0.8) + (0.20)(-0.1) + (-0.10)(0.3) = 0.175 - 0.400 - 0.020 - 0.030 = -0.275$
- $Z_{21}^1 = (-0.25)(0.5) + (0.40)(0.8) + (-0.30)(-0.1) + (0.15)(0.3) = -0.125 + 0.320 + 0.030 + 0.045 = 0.270$
- $Z_{31}^1 = (0.10)(0.5) + (0.05)(0.8) + (0.60)(-0.1) + (-0.45)(0.3) = 0.050 + 0.040 - 0.060 - 0.135 = -0.105$

- **Sample 2 (Column 2):**

- $Z_{12}^1 = (0.35)(-0.2) + (-0.50)(0.4) + (0.20)(0.9) + (-0.10)(-0.6) = -0.070 - 0.200 + 0.180 + 0.060 = -0.030$
- $Z_{22}^1 = (-0.25)(-0.2) + (0.40)(0.4) + (-0.30)(0.9) + (0.15)(-0.6) = 0.050 + 0.160 - 0.270 - 0.090 = -0.150$
- $Z_{32}^1 = (0.10)(-0.2) + (0.05)(0.4) + (0.60)(0.9) + (-0.45)(-0.6) = -0.020 + 0.020 + 0.540 + 0.270 = 0.810$

Therefore,

$$Z^1 = \begin{bmatrix} -0.275 & -0.030 \\ 0.270 & -0.150 \\ -0.105 & 0.810 \end{bmatrix}$$

Next, we apply the element-wise Rectified Linear Unit activation function:

$$\text{ReLU}(z) = \max(0, z)$$

Thus,

$$A^1 = \text{ReLU}(Z^1)$$

$$A^1 = \begin{bmatrix} \max(0, -0.275) & \max(0, -0.030) \\ \max(0, 0.270) & \max(0, -0.150) \\ \max(0, -0.105) & \max(0, 0.810) \end{bmatrix}$$

and therefore,

$$A^1 = \begin{bmatrix} 0.000 & 0.000 \\ 0.270 & 0.000 \\ 0.000 & 0.810 \end{bmatrix}$$

Notice how ReLU deactivates negative inputs by setting them to zero. This produces a sparse representation in the hidden layer. In this example, the non-zero activations are A_{21}^1 and A_{32}^1 .

2.2 Layer 2: Pre-activations Z^2 and Output A^2

The second layer receives the hidden-layer activations A^1 as input. The pre-activation matrix is computed as

$$Z^2 = W^2 A^1 + b^2$$

Substituting the numerical matrices:

$$Z^2 = \begin{bmatrix} 0.40 & -0.30 & 0.50 \\ -0.10 & 0.60 & -0.20 \end{bmatrix} \begin{bmatrix} 0.000 & 0.000 \\ 0.270 & 0.000 \\ 0.000 & 0.810 \end{bmatrix} + \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Evaluating each element:

$$Z_{11}^2 = (0.40)(0.000) + (-0.30)(0.270) + (0.50)(0.000) = -0.081$$

$$Z_{21}^2 = (-0.10)(0.000) + (0.60)(0.270) + (-0.20)(0.000) = 0.162$$

$$Z_{12}^2 = (0.40)(0.000) + (-0.30)(0.000) + (0.50)(0.810) = 0.405$$

$$Z_{22}^2 = (-0.10)(0.000) + (0.60)(0.000) + (-0.20)(0.810) = -0.162$$

Therefore,

$$Z^2 = \begin{bmatrix} -0.081 & 0.405 \\ 0.162 & -0.162 \end{bmatrix}$$

The output layer uses the Softmax activation function. For each sample (column), Softmax converts the logits into probabilities:

$$A_{ki}^2 = \frac{\exp(Z_{ki}^2)}{\sum_{j=1}^{n_2} \exp(Z_{ji}^2)}$$

For Sample 1:

$$A_{11}^2 = \frac{e^{-0.081}}{e^{-0.081} + e^{0.162}} \approx 0.4395$$

$$A_{21}^2 = \frac{e^{0.162}}{e^{-0.081} + e^{0.162}} \approx 0.5605$$

For Sample 2:

$$A_{12}^2 = \frac{e^{0.405}}{e^{0.405} + e^{-0.162}} \approx 0.6381$$

$$A_{22}^2 = \frac{e^{-0.162}}{e^{0.405} + e^{-0.162}} \approx 0.3619$$

Thus,

$$A^2 \approx \begin{bmatrix} 0.4395 & 0.6381 \\ 0.5605 & 0.3619 \end{bmatrix}$$

Each column sums to approximately 1, as expected for a Softmax output.

2.3 Loss Computation (Categorical Cross-Entropy)

For a mini-batch of $m = 2$ samples, the categorical cross-entropy loss is

$$L = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^{n_2} Y_{ki} \ln(A_{ki}^2)$$

Since the target labels are one-hot encoded,

$$Y = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

the loss becomes

$$L = -\frac{1}{2} [\ln(A_{11}^2) + \ln(A_{22}^2)]$$

Using the numerical values,

$$L = -\frac{1}{2} [\ln(0.4395) + \ln(0.3619)] \approx 0.948$$

Therefore, the initial mini-batch loss is approximately

$$\boxed{L \approx 0.948}$$

3 Step 2: Backward Pass & Chain Rule

3.1 Proof: Simplification of Softmax with Categorical Cross-Entropy

A common question during defense is why the gradient at the output layer simplifies to the straightforward subtraction $\delta^2 = A^2 - Y$.

Let z_i be the i -th pre-activation logit, $a_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$ the Softmax output, and Y_k the true one-hot target.

1. Derivative of Softmax ($\frac{\partial a_k}{\partial z_i}$)

Using the quotient rule on the Softmax definition:

- **Case $k = i$:**

$$\frac{\partial a_i}{\partial z_i} = \frac{e^{z_i}(\sum_j e^{z_j}) - e^{z_i}e^{z_i}}{(\sum_j e^{z_j})^2} = a_i(1 - a_i)$$

- **Case $k \neq i$:**

$$\frac{\partial a_k}{\partial z_i} = \frac{0 - e^{z_k}e^{z_i}}{(\sum_j e^{z_j})^2} = -a_k a_i$$

Combining both cases using the Kronecker delta δ_{ki} :

$$\frac{\partial a_k}{\partial z_i} = a_k(\delta_{ki} - a_i)$$

2. Applying the Chain Rule for Loss L

The Categorical Cross-Entropy loss for a single sample is $L = -\sum_k Y_k \ln(a_k)$. By applying the chain rule to compute $\delta_i = \frac{\partial L}{\partial z_i}$:

$$\delta_i = \frac{\partial L}{\partial z_i} = \sum_k \frac{\partial L}{\partial a_k} \frac{\partial a_k}{\partial z_i}$$

Since $\frac{\partial L}{\partial a_k} = -\frac{Y_k}{a_k}$:

$$\delta_i = \sum_k \left(-\frac{Y_k}{a_k} \right) [a_k(\delta_{ki} - a_i)]$$

Simplifying a_k in the numerator and denominator:

$$\delta_i = -\sum_k Y_k(\delta_{ki} - a_i) = -\sum_k Y_k \delta_{ki} + \sum_k Y_k a_i$$

3. Final Reduction

1. Since Y is a one-hot vector, $\sum_k Y_k = 1$. Therefore, $\sum_k Y_k a_i = a_i \sum_k Y_k = a_i$.
2. The Kronecker delta δ_{ki} isolates the term where $k = i$, so $\sum_k Y_k \delta_{ki} = Y_i$.

Substituting these two reductions back into the expression:

3.2 Proof: Derivation of Error Propagation (δ^l) and Bias Gradient ($\frac{\partial L}{\partial b^l}$)

To understand how errors flow backward through hidden layers, we apply the multivariable chain rule to the fundamental forward pass equations of any layer l :

$$z^l = W^l a^{l-1} + b^l, \quad a^l = \sigma(z^l)$$

where the local error vector is formally defined as $\delta^l = \frac{\partial L}{\partial z^l}$.

1. Derivation of the Bias Gradient (BP3)

We compute the partial derivative of the loss function L with respect to the bias vector b^l . Applying the chain rule:

$$\frac{\partial L}{\partial b^l} = \frac{\partial L}{\partial z^l} \cdot \frac{\partial z^l}{\partial b^l}$$

From the pre-activation equation $z^l = W^l a^{l-1} + b^l$, the derivative with respect to the bias term is simply the identity matrix ($\frac{\partial z^l}{\partial b^l} = 1$) because b^l is added directly without any scaling factor. Substituting $\delta^l = \frac{\partial L}{\partial z^l}$:

$$\boxed{\frac{\partial L}{\partial b^l} = \delta^l}$$

This proves that the gradient with respect to any bias vector is identically equal to the local error vector of that layer.

2. Derivation of Hidden Layer Error Projection (BP2)

For a single neuron j in hidden layer l , its pre-activation z_j^l influences the loss L through **all** k neurons in the subsequent layer $l + 1$. By the multivariable chain rule:

$$\delta_j^l = \frac{\partial L}{\partial z_j^l} = \sum_k \frac{\partial L}{\partial z_k^{l+1}} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l}$$

By definition, the error at neuron k in layer $l + 1$ is $\delta_k^{l+1} = \frac{\partial L}{\partial z_k^{l+1}}$. Substituting this into the summation gives:

$$\delta_j^l = \sum_k \delta_k^{l+1} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l}$$

Next, we evaluate the internal derivative $\frac{\partial z_k^{l+1}}{\partial z_j^l}$. Knowing that $z_k^{l+1} = \sum_m W_{km}^{l+1} a_m^l + b_k^{l+1}$ and $a_j^l = \sigma(z_j^l)$:

$$\frac{\partial z_k^{l+1}}{\partial z_j^l} = \frac{\partial z_k^{l+1}}{\partial a_j^l} \cdot \frac{\partial a_j^l}{\partial z_j^l} = W_{kj}^{l+1} \cdot \sigma'(z_j^l)$$

Substituting this derivative back into the sum yields:

$$\delta_j^l = \sum_k \delta_k^{l+1} \cdot W_{kj}^{l+1} \cdot \sigma'(z_j^l) = \left(\sum_k W_{kj}^{l+1} \delta_k^{l+1} \right) \cdot \sigma'(z_j^l)$$

3. Vectorization and Matrix Transposition

The scalar sum $\sum_k W_{kj}^{l+1} \delta_k^{l+1}$ computes the inner product using the column index j of matrix W^{l+1} . To express this in standard matrix-vector multiplication notation, we must transpose the weight matrix $(W^{l+1})^T$.

Extending the scalar result across all neurons in layer l using the Hadamard product ($\odot$) for element-wise multiplication by activation derivatives:

$$\delta^l = \left((W^{l+1})^T \delta^{l+1} \right) \odot \sigma'(z^l)$$

This proves that $(W^{l+1})^T$ acts as the backward mapping mechanism, distributing downstream errors proportionally to the weights that propagated them forward.

$$\boxed{\delta_i = a_i - Y_i} \quad \implies \quad \boxed{\delta^2 = A^2 - Y}$$

This elegant cancellation eliminates numerical instability during training and speeds up computation.

3.3 Output Layer Gradient (δ^2)

For a Softmax output combined with categorical cross-entropy loss, the gradient with respect to the pre-activation values is simplified to

$$\delta^2 = A^2 - Y$$

Therefore,

$$\delta^2 \approx \begin{bmatrix} 0.4395 & 0.6381 \\ 0.5605 & 0.3619 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\delta^2 \approx \begin{bmatrix} -0.5605 & 0.6381 \\ 0.5605 & -0.6381 \end{bmatrix}$$

3.4 Weight and Bias Gradients at Layer 2 (dW^2 , db^2)

The gradient with respect to the weights of Layer 2 is

$$dW^2 = \frac{1}{m} \delta^2 (A^1)^T$$

and the gradient with respect to the biases is

$$db^2 = \frac{1}{m} \sum_{i=1}^m \delta_{:i}^2$$

Thus,

$$dW^2 = \frac{1}{2} \begin{bmatrix} -0.5605 & 0.6381 \\ 0.5605 & -0.6381 \end{bmatrix} \begin{bmatrix} 0.000 & 0.270 & 0.000 \\ 0.000 & 0.000 & 0.810 \end{bmatrix}$$

which gives approximately

$$\begin{aligned} dW^2 &\approx \frac{1}{2} \begin{bmatrix} 0.000 & -0.1513 & 0.5168 \\ 0.000 & 0.1513 & -0.5168 \end{bmatrix} \\ &= \begin{bmatrix} 0.000 & -0.0757 & 0.2584 \\ 0.000 & 0.0757 & -0.2584 \end{bmatrix} \end{aligned}$$

For the bias gradient,

$$db^2 = \frac{1}{2} \begin{bmatrix} -0.5605 + 0.6381 \\ 0.5605 - 0.6381 \end{bmatrix}$$

and therefore

$$db^2 \approx \begin{bmatrix} 0.0388 \\ -0.0388 \end{bmatrix}$$

3.5 Hidden Layer Error Projection (δ^1)

The error is propagated from Layer 2 to Layer 1 using the chain rule:

$$\delta^1 = ((W^2)^T \delta^2) \odot \text{ReLU}'(Z^1)$$

where $\odot$ denotes element-wise multiplication.

The derivative of ReLU is

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

For our example,

$$\text{ReLU}'(Z^1) = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

First, we compute

$$(W^2)^T \delta^2 = \begin{bmatrix} 0.40 & -0.10 \\ -0.30 & 0.60 \\ 0.50 & -0.20 \end{bmatrix} \begin{bmatrix} -0.5605 & 0.6381 \\ 0.5605 & -0.6381 \end{bmatrix}$$

which gives approximately

$$(W^2)^T \delta^2 \approx \begin{bmatrix} -0.2803 & 0.3191 \\ 0.5045 & -0.5743 \\ -0.3924 & 0.4467 \end{bmatrix}$$

Applying Hadamard product with the ReLU derivative,

$$\delta^1 \approx \begin{bmatrix} 0 & 0 \\ 0.5045 & 0 \\ 0 & 0.4467 \end{bmatrix}$$

3.6 Weight and Bias Gradients at Layer 1 (dW^1 , db^1)

The gradient with respect to the Layer 1 weights is

$$dW^1 = \frac{1}{m} \delta^1 (A^0)^T$$

and the gradient with respect to the biases is

$$db^1 = \frac{1}{m} \sum_{i=1}^m \delta_{\cdot i}^1$$

Therefore,

$$dW^1 = \frac{1}{2} \begin{bmatrix} 0 & 0 \\ 0.5045 & 0 \\ 0 & 0.4467 \end{bmatrix} \begin{bmatrix} 0.5 & 0.8 & -0.1 & 0.3 \\ -0.2 & 0.4 & 0.9 & -0.6 \end{bmatrix}$$

which gives approximately

$$dW^1 \approx \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.1261 & 0.2018 & -0.0252 & 0.0757 \\ -0.0447 & 0.0893 & 0.2010 & -0.1340 \end{bmatrix}$$

For the biases,

$$db^1 = \frac{1}{2} \begin{bmatrix} 0 \\ 0.5045 \\ 0.4467 \end{bmatrix}$$

so that

$$db^1 \approx \begin{bmatrix} 0 \\ 0.2523 \\ 0.2234 \end{bmatrix}$$

4 Step 3: Adam Optimizer Step

Adam combines the advantages of momentum-based optimization and adaptive learning rates. For each parameter, Adam maintains a first moment estimate m_t and a second raw moment estimate v_t .

4.1 First and Second Moment Accumulation (m_t, v_t)

The first moment is updated according to

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

where g_t is the current gradient.

The second moment is updated according to

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

where the square is applied element-wise.

For the first Adam iteration, assuming

$$m_0 = 0, \quad v_0 = 0,$$

and using the standard values

$$\beta_1 = 0.9, \quad \beta_2 = 0.999,$$

we obtain

$$m_1 = 0.1 g_1$$

and

$$v_1 = 0.001 g_1^2.$$

4.2 Bias Correction ($\hat{m}_t, \hat{v}_t$)

Because the moment estimates are initialized at zero, they are biased toward zero during the first iterations. Adam compensates for this using bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

and

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

For $t = 1$,

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1} = \frac{0.1g_1}{0.1} = g_1$$

and

$$\hat{v}_1 = \frac{v_1}{1 - \beta_2} = \frac{0.001g_1^2}{0.001} = g_1^2.$$

Thus, during the first iteration, the bias-corrected moments reduce to

$$\boxed{\hat{m}_1 = g_1}, \quad \boxed{\hat{v}_1 = g_1^2}.$$

4.3 Final Parameter Update

The Adam parameter update is

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where

- α is the learning rate,
- ϵ is a small constant used for numerical stability,
- $\hat{m}_t$ is the bias-corrected first moment,
- $\hat{v}_t$ is the bias-corrected second moment.

For example, using

$$\alpha = 0.001, \quad \epsilon = 10^{-8},$$

the updated parameters are obtained independently for every element of W^1 , b^1 , W^2 , and b^2 :

$$W_{\text{new}}^1 = W^1 - \alpha \frac{\hat{m}_{W^1}}{\sqrt{\hat{v}_{W^1}} + \epsilon}$$

$$b_{\text{new}}^1 = b^1 - \alpha \frac{\hat{m}_{b^1}}{\sqrt{\hat{v}_{b^1}} + \epsilon}$$

$$W_{\text{new}}^2 = W^2 - \alpha \frac{\hat{m}_{W^2}}{\sqrt{\hat{v}_{W^2}} + \epsilon}$$

$$b_{\text{new}}^2 = b^2 - \alpha \frac{\hat{m}_{b^2}}{\sqrt{\hat{v}_{b^2}} + \epsilon}.$$

These updated parameters constitute the starting point for the next training iteration.